La Plateforme
La grande école du numérique pour tous

Les bases de Java - jour 1

Un peu d'histoire

juin 1991

né sous la forme d'un projet appelé "**Oak**" travaillant pour Sun Microsystems.

1995

nouveau nom : **Javain** en raison du fait que Oak était déjà enregistré dans le cadre d'une autre marque.
Son **développement** n'a été **terminé** qu'en **1995**.

1996

Sun Microsystems a publié la **première implémentation publique** de Java 1.0.

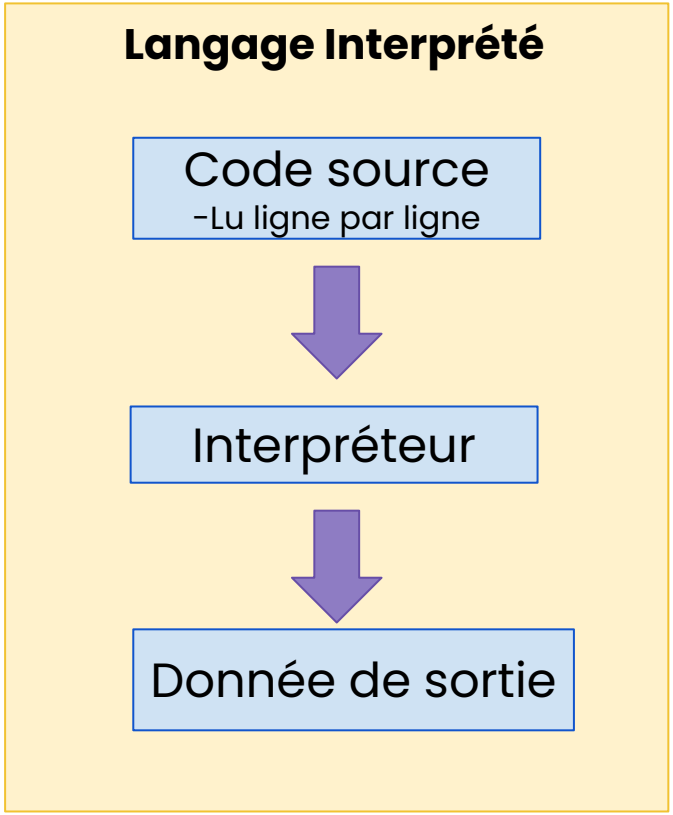
1998

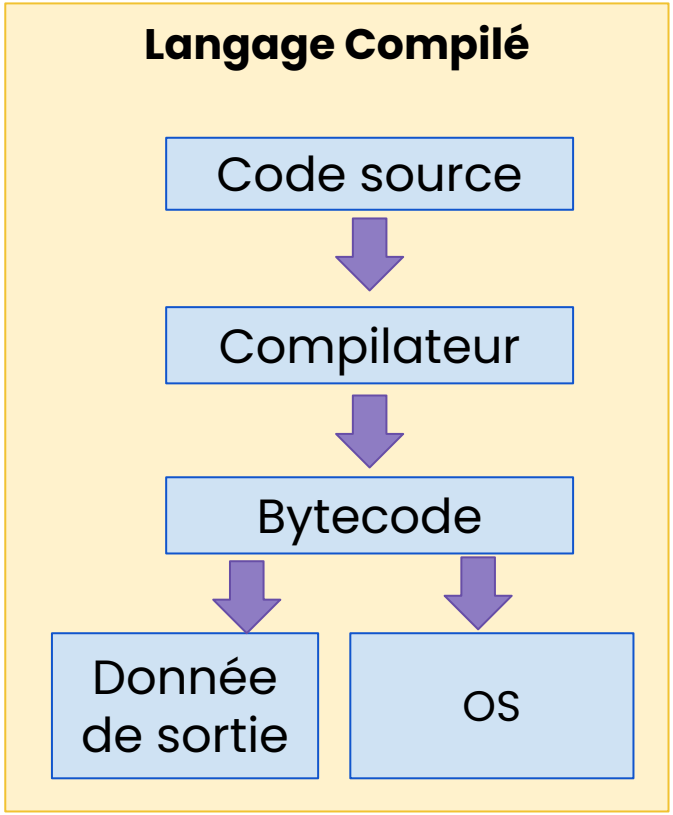
Java 2 a ajouté plusieurs configurations conçues pour **différents types de plates-formes.**

2017

une nouvelle version lancée tous les six mois, afin de s'assurer que les technologies liées à Java sont mises à jour en temps opportun en fonction des **besoins et des exigences du marché actuel.**

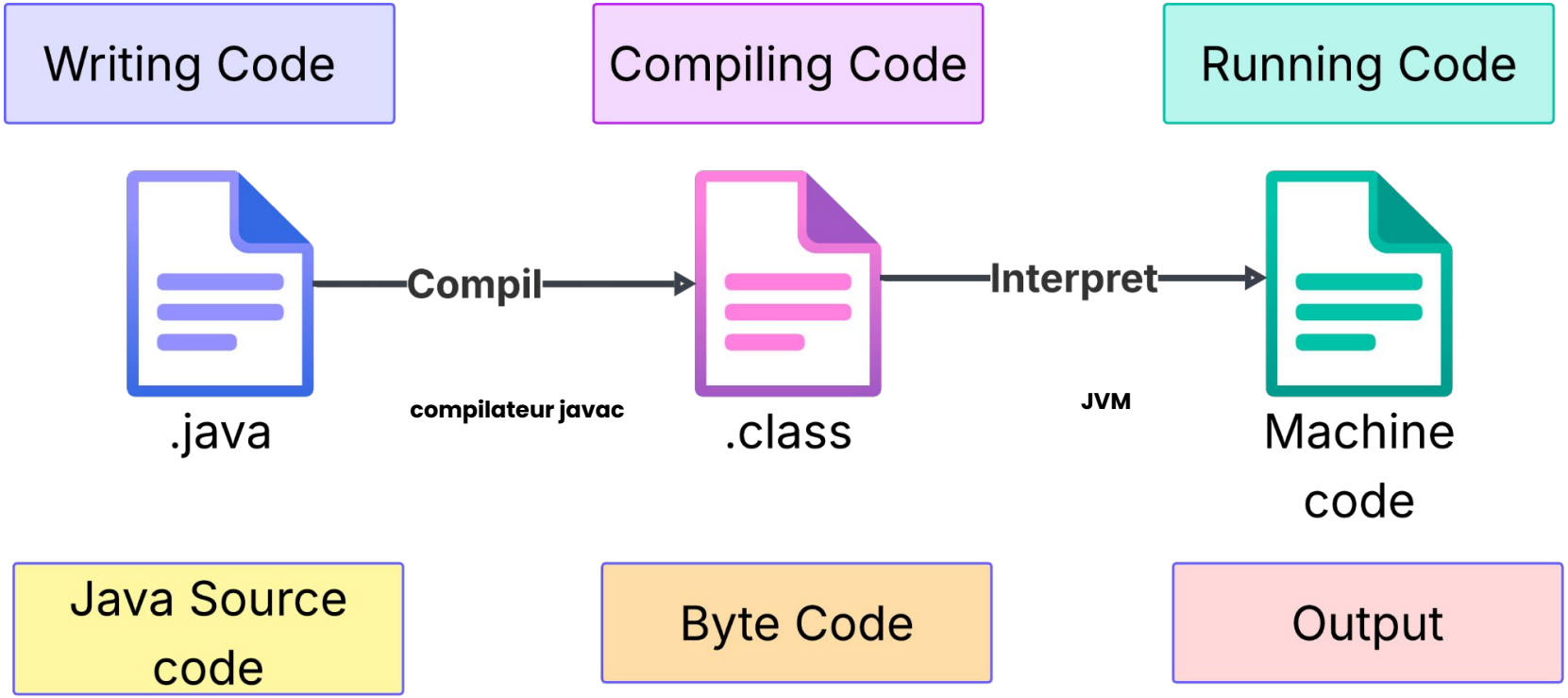
Interprété / Compilé





Le cas de Java

compile once, run anywhere



/ A quoi ça sert ?

- Langage de programmation **rapide**;
- Réputé pour sa **fiabilité**;
- Orienté **objet**;
- première technologie dont la **sécurité** fait partie intégrante de la conception;
- **Multithread**;
- Permet de tout coder : des applications **mobiles** aux **logiciels** d'entreprise en passant par les **applications** de **big data** et les technologies **côté serveur**.

Commencer un projet

Installation du kit de développement Java (JDK) sur votre ordinateur.

1

JDK

Configuration de votre IDE

2

IDE

Création d'un nouveau projet Java dans votre IDE.

3

Création

Développement

4

Écriture du code source pour votre application, en utilisant les bibliothèques et les API nécessaires.

Compilation

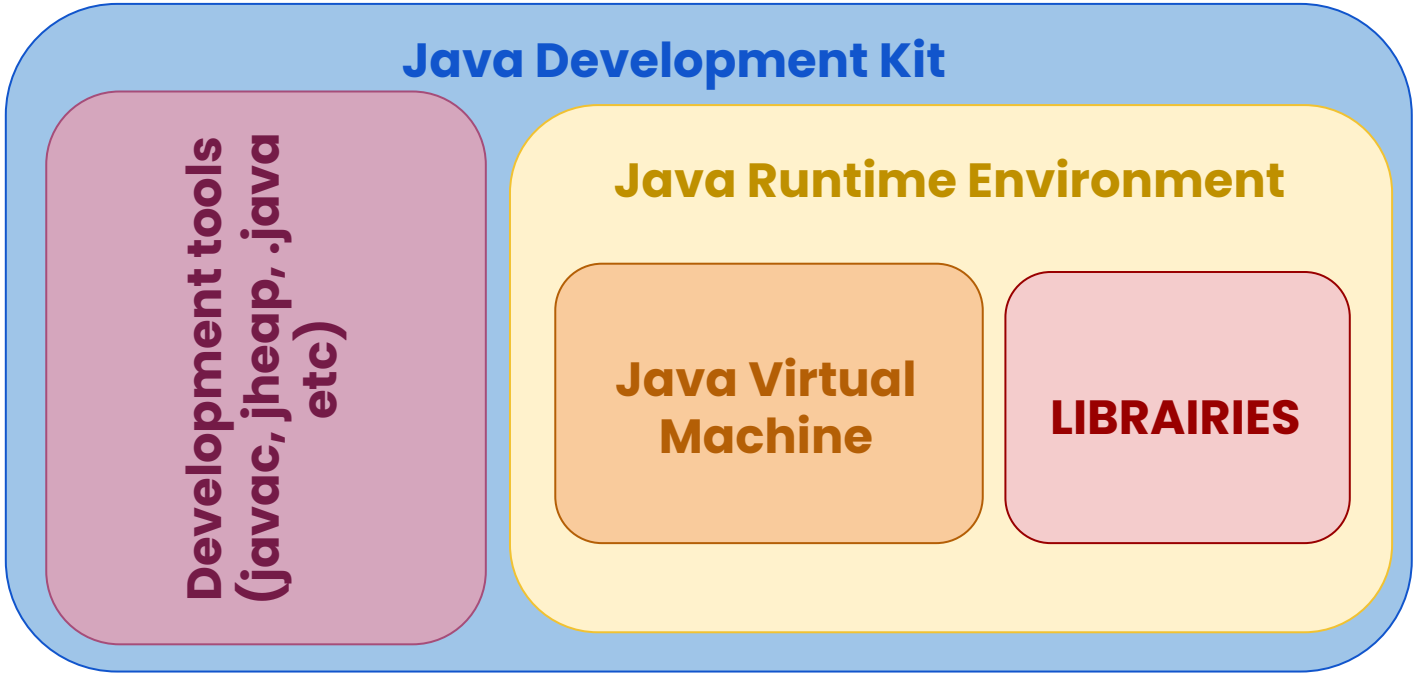
5

Compilation de votre code source en bytecode, qui peut être exécuté sur n'importe quelle plateforme Java

JDK : désigne un ensemble de bibliothèques logicielles de base du langage de programmation Java, ainsi que les outils avec lesquels le code Java peut être compilé, transformé en bytecode destiné à la machine virtuelle Java.

JRE : environnement d'exécution Java, est un ensemble d'outils logiciels responsables de l'exécution du programme ou de l'application Java sur votre système.

JVM est responsable de la conversion du bytecode en code spécifique à la machine et est nécessaire dans JDK et JRE.



Ecrire du Java

Nom classe porte le nom du fichier

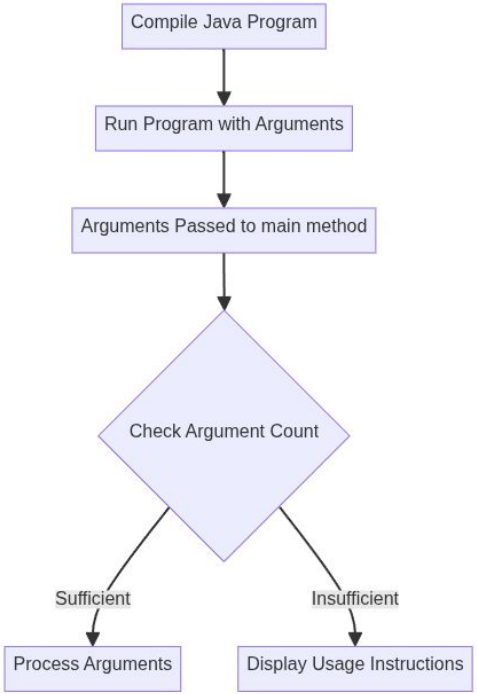
Tableau de chaînes de caractères : contient les **arguments passés en ligne de commande** au lancement du programme.

Fonction main (point d'entrée)

```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("hello world");
    }
}
```

Instructions

Les arguments Java



flux de traitement des arguments

L'exécution d'applications Java **avec** des **arguments** offre aux développeurs un **mécanisme puissant** pour **configurer** et **contrôler dynamiquement** le **comportement** d'un programme.

- Les **arguments** sont toujours passés sous forme de **chaînes de caractères;**
- Le premier argument est à **l'index 0;**
- Toujours **valider** et **gérer** les arguments de **manière sécurisée;**
- Utiliser une **conversion** de type **appropriée** si nécessaire.

Compilation et exécution

Compilation

Exécution

Résultat

```
javac HelloWorld.java
java HelloWorld
Hello world
```

/ Types et variables

- langage **fortement typé**.
- **camelCase**
- Chaque **variable** doit être **typée à sa création**.
- Une **variable**, une fois créée, ne peut **plus changer de type**.

Initialisation

Types primitifs :

byte : byte b = 10;

short : short s = 200;

int : int i = 1000;

long : long l = 100000L;

float : float f = 3.14f;

double : double d = 3.14159;

char : char c = 'A';

boolean : boolean b = true;

Types de référence :

String : String s = "Hello, world!";

Object : Object o = new Object();

Array : int[] arr = {1, 2, 3};

Class : Class<?> cls = String.class;

Interface : Runnable r = () -> System.out.println("Hello, world!");

ne stockent **pas directement** les **valeurs des objets** mais plutôt des **références (adresses) pointant** vers les **objets** dans la mémoire.

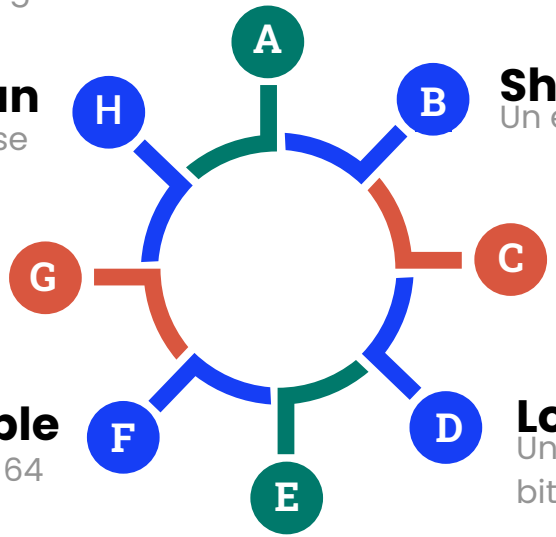
Min & Max

Types	Min	Max
Byte	-128	127
Short	-32768	32767
Int	-2147483648	2147483647
Long	-9223372036854775808	9223372036854775807
Float	- 3.40282347 x 10^38	3.40282347 x 10^38
Double	- 1.7976931348623157 x 10^308	1.7976931348623157 x 10^308

Différences type primitif vs référence

	types primitifs	types référence
Stockage	**stockent directement** leurs valeurs dans la mémoire.	**stockent** des **adresses** pointant vers des objets situés dans la mémoire dynamique (heap).
Allocation de mémoire	généralement alloués dans la **pile** (**stack**).	alloués dans le **tas** (**heap**).
Passage de paramètres	**passés par valeur.** Une copie de la valeur est passée à la méthode.	**passés par valeur de la référence.** La copie de l'adresse mémoire est passée à la méthode, permettant à la méthode de manipuler l'objet référencé.

Les différents types



Byte
Un entier signé sur 8 bits

Short
Un entier signé sur 16 bits

Int
Un entier signé sur 32 bits

Long
Un nombre à virgule flottante sur 64 bits

Float
Un nombre à virgule flottante sur 32 bits

Double
Un nombre à virgule flottante sur 64 bits

Char
Un caractère Unicode sur 16 bits

Boolean
une valeur booléenne, soit true ou false

/ Syntaxe

- Contrairement au Python, et comme le C/C++ les **blocs** sont définis par des **accolades**.
- Chaque **instruction** se **termine** par **“;”** : cela permet au programme de mieux comprendre ce qu’on lui demande.

/ Fonctions

- Les **fonctions** sont elles aussi **typées**.
- Les **paramètres** d'une fonction doivent aussi être **typés**.

/ Qu’est-ce qu’un programme ?

- Un programme en Java doit posséder **une fonction** nommée **“main”.**
- Quand vous le lancez, votre programme **commence son exécution** par la fonction main.
- On appelle le main “**point d'entrée**”.

La Plateforme
La grande école du numérique pour tous

Les boucles

Boucles : for

```java
public class PrintNumber {
    public static void main(String[] args) {
        for (int number = 1; number ≤ 10; number++) {
            System.out.println("Valeur de number : " + number);
        }
    }
}
```

Avantages :

- **Clarté et concision :** connaissance en **amont** du **nombre d'itérations** à effectuer.
  Regroupe initialisation + condition de continuation + incrémentation en une seule ligne.

- **Facile à comprendre.**

Inconvénients :

- **Moins flexible :** pour les scénarios où le nombre d'itérations n'est pas connu à l'avance ou dépend de conditions dynamiques complexes.

- **Complexité accrue pour des structures plus élaborées**

Boucles : do while

```java
public class PrintNumber {
    public static void main(String[] args) {
        int number = 1;
        do {
            System.out.println("Valeur de number : " + number);
            number++;
        } while (number <= 10);
    }
}
```

Avantages :

- **Exécution garantie du bloc au moins une fois :** garantit que le corps de la boucle est **exécuté au moins une fois**, même si la condition est fausse dès le départ. Utile pour les situations où une action doit être effectuée au moins une fois avant toute vérification de condition.

- **Utilisation simple pour certains scénarios :** action doit précéder la condition de vérification

Inconvénients :

- **Moins couramment utilisée** : En raison de sa syntaxe, moins couramment utilisée.

- **Risque de boucles infinies.**

Boucles : while

```java
public class PrintNumber {
    public static void main(String[] args) {
        int number = 1;
        while (number ≤ 10) {
            System.out.println("Valeur de number : " + number);
            number++;
        }
    }
}
```

Avantages :

- **Grande flexibilité** : utilisée dans de nombreux scénarios, en particulier lorsque le **nombre d'itérations n'est pas connu à l'avance**.
- **Simple pour les conditions dynamiques :** Idéale pour les situations où la continuation dépend de conditions dynamiques ou complexes qui changent au cours de l'exécution.

Inconvénients :

- **Risques de boucles infinies**
- **Séparation de l'initialisation et de l'incrémentation :** l'initialisation et l'incrémentation sont séparées de la déclaration de la boucle, ce qui peut rendre le code moins compact et moins lisible pour des boucles simples avec un compteur.

La Plateforme
La grande école du numérique pour tous

Les instructions conditionnelles

instruction conditionnelle : if/else if/else

```java
public class AgeCategory {
    public static void main(String[] args) {
        int age = 25;

        if (age < 13) {
            System.out.println("Enfant");
        } else if (age < 18) {
            System.out.println("Adolescent");
        } else if (age < 65) {
            System.out.println("Adulte");
        } else {
            System.out.println("Senior");
        }
    }
}
```

Avantages :

- **Flexibilité :** tests de conditions complexes, combinant différentes expressions logiques
- **Pas de restriction sur les types**
- **Nesting :** Peut imbriquer d'autres conditions if à l'intérieur, permettant des structures de décision complexes.

Inconvénients :

- **Lisibilité**
- **Performance :** Si beaucoup de conditions doivent être évaluées, cela peut être moins performant que certaines alternatives.

instruction conditionnelle : switch

```java
public class DayOfWeek {
    public static void main(String[] args) {
        int day = 3;

        switch (day) {
            case 1:
                System.out.println("Lundi");
                break;
            case 2:
                System.out.println("Mardi");
                break;
            case 3:
                System.out.println("Mercredi");
                break;
            case 4:
                System.out.println("Jeudi");
                break;
            case 5:
                System.out.println("Vendredi");
                break;
            case 6:
                System.out.println("Samedi");
                break;
            case 7:
                System.out.println("Dimanche");
                break;
            default:
                System.out.println("Jour invalide");
                break;
        }
    }
}
```

Avantages :

- **Lisibilité et Organisation**
- **Performance :** notamment lorsque le compilateur peut optimiser les sauts directs dans les cas de valeurs constantes (**table de saut**).
- **Code plus propre**

Inconvénients :

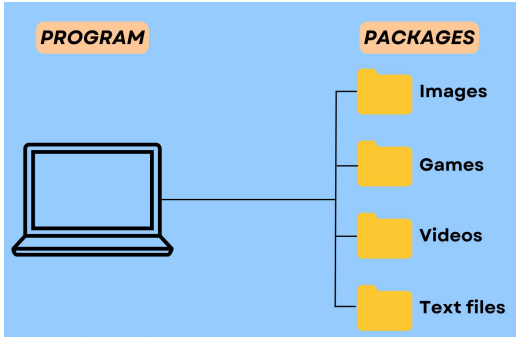
- **Types limités :** avant Java 7 ne permettent de tester que les types int, char, short, byte, ou leurs équivalents boxed (Integer, Character). Depuis Java 7, le switch supporte aussi les Strings.
- **Pas de conditions complexes :** Vous ne pouvez comparer qu'une seule variable contre des valeurs constantes.
- **Oublis de break**

La Plateforme

La grande école du numérique pour tous

package

package en Java



utilisation du mot clé package suivit du nom de votre package.

définition doit être la première ligne de code effective de votre fichier.

```java
package hello;

public class Demo {
    // TODO
}
```

```java
package fr.koor.hello;

public class Demo {
    // TODO
}
```

normalement, être écrit totalement en minuscules (pas de lettres majuscules)

peut contenir des sous packages : chaque sous-package est séparé de son parent avec un point. Il n'y a pas de limite sur le niveau de profondeur.

Des Questions ?

La Plateforme